Methodology

Methodology I

This section describes the implementation methodology, explicitly detailing how the optimization algorithms are constructed, validated, and analyzed using the Generalized Research Template's `infrastructure` and `tests` ecosystems.

Algorithm Implementation

Gradient Descent Algorithm I

The core algorithm implements the iterative procedure for unconstrained optimization. The `optimizer` module uses the standard-library `logging` logger for optional verbose diagnostics; the analysis orchestrator uses `infrastructure.core.logging.utils.get_logger`. Tests run under the hermetic boundaries defined in the test configuration.

Algorithm — Gradient Descent (implemented in the optimizer module)

Input: *Initial point* x_0*, step size* α*, tolerance* ϵ*, max iterations* $N_{\max}$

1. *Initialize* $k \leftarrow 0$
2. ***While*** $k < N_{\max}$ ***do:***
   - *Compute gradient* $g_k = \nabla f(x_k)$
   - ***If*** $\|g_k\|_2 < \epsilon$ ***then return*** x_k *(converged)*
   - *Update:* $x_{k+1} \leftarrow x_k - \alpha \cdot g_k$
   - $k \leftarrow k + 1$
3. ***Return*** x_k *(max iterations reached)*

Infrastructure Integration

Infrastructure Integration I

The methodology explicitly bridges theoretical mathematics with production-grade validation through the `infrastructure.scientific` module.

Numerical Stability Analysis I

Rather than writing ad-hoc validation code, the project imports `infrastructure.scientific.stability.check_numerical_stability`. This utility subjects the objective function to a barrage of extreme inputs (NaN, Inf, $\pm 10^{10}$) to calculate a formalized stability score. If this score degrades, the analysis orchestrator execution deliberately aborts, ensuring the methodology cannot enter unrecoverable states.

Performance Benchmarking I

The analysis exercises `infrastructure.scientific.benchmarking.benchmark_function` as a runtime diagnostic. Host-dependent timing and memory observations are logged but excluded from tracked evidence. The canonical report records fixed inputs and exact objective values, while the dimensional figure reports deterministic convergence iterations and the proxy $d \times$ iterations; neither surface claims that one host run proves an asymptotic bound.

Convergence Analysis

Convergence Analysis I

For quadratic functions $f(x) = \frac{1}{2}x^T Ax - b^T x$ where A is positive definite, the convergence factor becomes (Bertsekas 1999):

$$\rho = \frac{|\lambda_{\max} - \alpha\lambda_{\min}|}{|\lambda_{\min} + \alpha\lambda_{\max}|} \tag{1}$$

Optimal convergence occurs when $\alpha = \frac{2}{\lambda_{\min}+\lambda_{\max}}$, yielding $\rho = \frac{\kappa-1}{\kappa+1}$.

Experimental Setup

Step Size Analysis I

Step sizes are not chosen ad hoc in the manuscript: they are read from `experiment.step_sizes` in `manuscript/config.yaml` and passed through `run_convergence_experiment()` in `src/template_code_project/analysis/` (entry: `scripts/optimization_analysis.py`). The active grid for this build is:

Step Size Analysis I

- $\alpha = 0.01$ (conservative)
- $\alpha = 0.1$ (conservative)
- $\alpha = 0.5$ (near-optimal)
- $\alpha = 1.0$ (near-optimal)
- $\alpha = 1.5$ (aggressive)
- $\alpha = 2.5$ (divergent (expected unstable for H = I))

Step Size Analysis I

Labels follow the same agency taxonomy used for plot colours (`agency_category` in `template_code_project.figures.viz_config`): **conservative** (small α, slow but stable on $H = I$), **near-optimal** (including $\alpha = 1$ where the method reaches x^* in one step for this quadratic), **aggressive** ($1 < \alpha < 2$, still linearly contracting in $|x - x^*|$ but oscillatory in sign), and **divergent** ($|1 - \alpha| \geq 1$).

Zero-Mock Testing Methodology I

The most critical aspect of the project's methodology is its validation framework. The project is governed by a strict Zero-Mock testing policy, evaluated actively by executing `uv run pytest projects/templates/template_code_project/tests/` during the infrastructure build phase (negative control: the repository lexical no-mock gate `scripts/audit/verify_no_mocks.py --inventory` flags any prohibited mock import or call and fails on dependency-replacement debt).

Zero-Mock Testing Methodology I

1. **Project tests**: the project optimizer test suite exercises `src/template_code_project/core/optimizer.py` (typical, edge, boundary, and pathological inputs including NaN/Inf and zero gradients) and, when infrastructure imports succeed, call into `optimization_analysis.py` helpers—without mocks. Suite size: the measured repository counts.
2. **Infrastructure validation**: The repository-level `tests/infra_tests/` suite validates shared template modules (e.g. pipeline and discovery helpers) independently of this project's manuscript.
3. **Coverage Gates**: The GitHub Actions CI workflow enforces a mandatory $\geq$90% statement coverage gate on `projects/templates/template_code_project/src/` prior to treating the project as build-green. Coverage below the floor fails the test process through `--cov-fail-under`, so build-green status cannot be reached while silently skipping weakened coverage. Falling below that floor fails the gate outright (`--cov-fail-under` enforces it).

Stopping rule and reporting I

`gradient_descent()` terminates when $\|\nabla f(x_k)\|$ falls below `experiment.tolerance`, when k reaches `experiment.max_iterations`, or when an objective, gradient, or update becomes non-finite. The boolean `converged` plus `termination_reason` in exported CSV rows distinguish these outcomes. A non-finite candidate is rejected before it enters `objective_history`, preserving the last finite state for reproducible reporting. Downstream, `scripts/z_generate_manuscript_variables.py` aggregates the CSV into `RESULT_*` placeholders so tables and prose cannot drift from the last analysis run.

Figure generation contract I

Each figure in `03_results.md` maps to a generator in `src/template_code_project/figures/` (`generate_convergence_plot`, `generate_step_size_sensitivity_plot`, `generate_convergence_rate_plot`, `generate_complexity_visualization`, `generate_stability_visualization`, `generate_benchmark_visualization`), orchestrated by `src/template_code_project/analysis/` / `scripts/optimization_analysis.py`. Captions in the markdown intentionally name the function and the key parameters (tolerance lines, grids, dimensions) so reviewers can navigate from PDF to code without inferring hidden defaults.

Figure generation contract I

The same generator writes explicit `metadata.alt_text` for every registered figure. The alt text states what the visual encodes and preserves the boundary between deterministic observations and host-dependent diagnostics. Run the opt-in publication migration gate with `--require-figure-accessibility` to verify that every referenced figure has this metadata before promotion.

Analysis Pipeline & LaTeX Integration

Analysis Pipeline & LaTeX Integration I

The automated analysis script leverages `infrastructure.core.progress` (`ProgressBar`, `SubStageProgress`) to orchestrate experiments, collect convergence trajectories, and generate publication-quality visualizations seamlessly.

The research template supports advanced LaTeX customization through the `preamble.md` configuration. This is ingested directly by `infrastructure.rendering.latex_utils.py` and `pdf_renderer.py`, automatically linking compiled PGF plots and BibTeX citations. This automated approach ensures an unbreakable chain of custody from raw algorithmic execution to the final rendered manuscript.